

Architectural Analysis of Enterprise Video Conferencing Platforms and Next-Generation Deployment Strategies on Vercel and Supabase

The landscape of real-time communication (RTC) has undergone a profound transformation over the last decade, driven by the escalating demands for global, low-latency, and highly concurrent video conferencing architectures. Enterprise platforms such as Google Meet, Zoom, Microsoft Teams, and WhatsApp utilize deeply complex, highly distributed systems to manage media routing, state synchronization, end-to-end encryption, and network resilience. Replicating this enterprise-grade infrastructure on modern serverless and edge-computed stacks—specifically utilizing Vercel and Supabase—requires a nuanced, comprehensive understanding of WebRTC protocols, Selective Forwarding Units (SFUs), ephemeral edge computing environments, and reactive state management paradigms. This report dissects the underlying codebases, programming principles, and feature integrations of premium RTC platforms, providing a blueprint for achieving structural parity using contemporary web application frameworks.

Core Topologies in Real-Time Communication

At the foundational level of any video conferencing platform lies the media routing topology. The architectural choice deployed at this layer dictates the ultimate scalability, bandwidth consumption, and server-side computational requirements of the entire application. The underlying transport mechanism for virtually all modern RTC is WebRTC, an open-source framework enabling direct real-time communication via JavaScript APIs.¹ WebRTC relies on a sophisticated suite of networking protocols to traverse firewalls and Network Address Translators (NATs). This traversal is achieved using the Interactive Connectivity Establishment (ICE) framework, which utilizes Session Traversal Utilities for NAT (STUN) servers to discover public IP addresses, and Traversal Using Relays around NAT (TURN) servers as fallbacks to relay traffic when direct peer-to-peer connections are blocked by strict enterprise firewalls.³

The most rudimentary routing approach built upon WebRTC is the Peer-to-Peer (P2P) mesh network. In a P2P architecture, every participant in a call establishes a direct WebRTC `RTCPeerConnection` with every other participant. This eliminates the need for centralized media servers, thereby ensuring absolute minimal latency and reducing backend infrastructure costs.¹ However, the required uplink bandwidth scales exponentially with the number of participants. A client in a mesh network must upload $n - 1$ distinct media streams for a call with n participants, fundamentally restricting scalability and rapidly degrading client-side

performance beyond three to five users.⁵

To circumvent the inherent limitations of mesh networks, legacy enterprise solutions initially adopted the Multipoint Conferencing Unit (MCU) architecture. An MCU operates as a centralized mixing server that receives encoded video streams from all participants, decodes them, composites them into a single continuous, unified video feed, re-encodes the composite stream, and transmits it back to the clients.⁵ While this approach strictly minimizes client-side bandwidth and processing requirements—as each client only uploads one stream and downloads one stream—it imposes an extraordinary computational burden on the server infrastructure. The constant, parallel decoding and re-encoding processes generate exorbitant infrastructure costs and introduce inevitable processing latency, rendering it inefficient for modern cloud-native deployments.⁵

Consequently, the industry standard for modern enterprise platforms, including Zoom and Google Meet, is the Selective Forwarding Unit (SFU) architecture.⁵ An SFU receives encrypted media streams from each participant and intelligently routes, or "forwards," these streams to the appropriate receiving participants without decoding or altering the underlying media payload.⁵ This model effectively shifts the decoding burden back to the robust processors of modern client devices while dramatically reducing upstream bandwidth requirements, as each user only uploads their stream once to the centralized server.⁷ The SFU dynamically adapts to fluctuating network conditions by selectively dropping packets or switching stream qualities based on the specific receiving capacity of individual clients.⁶

Architectural Model	Data Flow Mechanism	Server Processing Load	Client Bandwidth Requirement	Scalability
Peer-to-Peer (Mesh)	Direct client-to-client connections	Zero (No media server required)	Exponential ($n - 1$ upstreams)	Extremely Low (Max 3-5 users)
Multipoint Conferencing Unit (MCU)	Server decodes, mixes, and re-encodes streams	Extremely High	Constant (1 upstream, 1 downstream)	Moderate (Server-bound)

Selective Forwarding Unit (SFU)	Server forwards streams without decoding	Low to Moderate	Linear downstream, 1 upstream	Extremely High
--	--	-----------------	-------------------------------	----------------

Encoding Strategies: Simulcast versus Scalable Video Coding

Within the SFU paradigm, platforms employ advanced encoding strategies to cater to heterogeneous network environments. The two dominant programming strategies for managing this are Simulcast and Scalable Video Coding (SVC).

Simulcast represents a brute-force but highly compatible approach where the transmitting client's WebRTC implementation simultaneously encodes its video feed into multiple independent resolutions and bitrates (for instance, 1080p, 720p, and 360p).⁴ The SFU acts as a sophisticated traffic controller, forwarding the high-quality stream to participants with robust connections and seamlessly switching to the lower-quality streams for participants experiencing network degradation or viewing the stream in a smaller UI tile.¹⁰ While highly effective and broadly supported across legacy browsers, Simulcast demands massive CPU resources from the transmitting device to handle multiple concurrent encoding pipelines, and it results in inherently redundant data transmission.¹⁰

Scalable Video Coding (SVC) provides a far more mathematically elegant, layered approach. Under SVC, the transmitting client encodes a single video stream composed of a foundational "base layer" (representing the lowest quality) accompanied by multiple sequential "enhancement layers".⁴ If a receiving client possesses sufficient bandwidth, the SFU forwards the base layer along with all available enhancement layers, which the client decoder combines to render high-resolution video.¹⁰ If network conditions deteriorate, the SFU intelligently strips away the enhancement layers, leaving the resilient base layer intact to prevent the stream from dropping.¹⁰ This methodology eliminates redundant data transmission and significantly lowers uplink bandwidth requirements, though it introduces high complexity into the SFU logic, as the server must deeply parse the RTP packets to understand layer dependencies and route them correctly without breaking the client's decoding chain.¹⁰

Deep Architectural Analysis of Enterprise Platforms

Google Meet: Virtual Media Streams and Codec Optimization

Google Meet's underlying architecture demonstrates an extreme, highly orchestrated optimization of the SFU model through the implementation of Virtual Media Streams. In massive enterprise conferences, transmitting every participant's stream to a single client would

catastrophically overwhelm local decoding capabilities and memory heaps. Instead, Google Meet's SFU multiplexes active and relevant participant streams onto a fixed, significantly smaller set of outgoing virtual streams.⁹

This architectural abstraction requires meticulous handling of WebRTC primitives. A standard Real-time Transport Protocol (RTP) connection relies on a Synchronization Source (SSRC), a 32-bit identifier denoting a single, specific media stream.⁹ In Google Meet, the client application establishes exactly three audio transceivers and a limited number of video transceivers (ranging from one to three).⁹ The SSRCs assigned to these specific transceivers remain entirely static throughout the duration of the session, an anomaly compared to standard WebRTC implementations.⁹ To identify the actual human speaker occupying a virtual stream at any given millisecond, Google Meet utilizes the Contributing Source (CSRC) identifier located within the RTP packet header.⁹ The CSRC is dynamically injected by the SFU and represents the unique identifier assigned to a user upon initially joining the call.⁹ The client application continuously parses the CSRC to map incoming media to the correct DOM elements and user interface tiles in real-time.

Furthermore, Google Meet mandates extraordinarily strict performance constraints on its underlying video codecs. The platform utilizes VP8, VP9, and AV1. For AV1 deployments, decoders are subjected to rigorous throughput validations, requiring the capacity to simultaneously process multiple 2880×1880 streams at 30 frames per second without violating maximum inter-frame processing times of $1 / \max(30, \text{fps})$ seconds.⁹ If the requested resolution cannot be met, the SFU dynamic scaling logic applies temporal and spatial scalability protocols to step down the quality rather than dropping the connection.⁹

Google Meet also leverages powerful machine learning integrations directly within the WebRTC pipeline. Features such as background blur and virtual background replacements do not rely heavily on cloud servers; instead, they execute locally utilizing CPU-optimized TensorFlow Lite or WebAssembly (WASM) implementations of segmentation models like MediaPipe or DeepLabv3.¹⁴ By running these models on the client device, Google eliminates the latency and cloud compute costs associated with round-trip image processing. For noise cancellation, Google employs a sophisticated hybrid approach: processing can occur via client-embedded noise suppression modules (e.g., Rust implementations targeting Emscripten-backed WASM) to avoid per-minute cloud filtering costs, or it can be securely offloaded to specialized cloud Tensor Processing Units (TPUs) for complex, arbitrary noise profiles requiring advanced neural network inference.¹⁷

Beyond media routing, Google Meet incorporates deep integration with the Google Workspace ecosystem through the Google Meet Add-ons SDK.¹⁹ This SDK allows developers to embed their applications directly into the Meet interface using an architecture based on iframe loading and explicit AddonSession instantiation.²⁰ The integration allows plugins to surface either on the main stage (replacing the primary video feed) or within the side panel.²¹

State synchronization across these plugins is handled via the Co-Doing API, allowing real-time collaboration with programmable slack thresholds to prevent buffering loops when participants' streams are slightly out of sync.²³ Furthermore, chat synchronization is managed continuously, writing real-time meeting chat data back to persistent Google Chat spaces for internal users, while ensuring ephemeral destruction of chat histories for external guests once the meeting concludes.²⁵

Zoom: Distributed Topologies and Component Decoupling

Zoom's architecture is a heavily distributed, proprietary framework that deliberately separates signaling, session control, and media routing. The architecture relies on globally distributed Multimedia Routers (MMRs) and Zone Controllers.²⁶ A cluster of these servers creates a functional "Meeting Zone." The Zone Controller handles session management, participant rosters, and WebSocket-based signaling, while the MMR functions similarly to an optimized SFU, aggressively routing UDP packets based on geographic proximity to minimize latency.⁸

A defining characteristic of Zoom's engineering is its implementation of dynamic feature subsets, most notably Breakout Rooms. Architecturally, a breakout room is not merely a visual client-side DOM manipulation; it requires a full, programmatic reallocation of the participant's media and signaling channels.²⁷ When a host initiates breakout rooms, the Zone Controller signals the clients to gracefully tear down their current RTP transceivers and establish new connections bound to a dynamically provisioned virtual sub-room. The system orchestrates state synchronization across these discrete rooms, allowing the host to broadcast text messages globally or observe status indicators across deeply separated architectural boundaries.²⁷

Zoom's database and storage stack reflects a microservices philosophy optimized for different data schemas. Unstructured data, such as chat messages and meeting telemetry, are stored in highly available NoSQL databases like MongoDB or Cassandra.²⁶ Video recordings are transcoded and pushed to object storage solutions like Amazon S3, which interface with localized Content Delivery Networks (CDNs) for high-speed playback.²⁶ For real-time state access, the architecture makes extensive use of in-memory caching systems like Redis or Memcached.²⁶

Integration with external ecosystems, such as Google Calendar and Microsoft Outlook, is achieved through sophisticated background polling and OAuth 2.0 authorization flows.²⁹ Once authorized, Zoom sets up asynchronous webhooks to listen for event creation or deletion on the third-party calendar, ensuring bi-directional synchronization.³¹ For developers extending Zoom's capabilities, the Zoom Video SDK provides low-level access to the media pipeline. Initialization of this SDK relies on the programmatic generation of JSON Web Tokens (JWTs) containing critical payload data including the app_key, tpc (session name), role_type (host vs participant), and strict issue and expiration timestamps (iat and exp) to ensure cryptographic session validity.³²

WhatsApp: End-to-End Encrypted Multiparty Routing

While enterprise platforms like Zoom and Google Meet focus on massive concurrent scale, WhatsApp prioritizes strict privacy through comprehensive End-to-End Encryption (E2EE) utilizing the Signal Protocol.³³ Translating E2EE to a multiparty video routing topology introduces immense cryptographic overhead, as the central SFU cannot decrypt, inspect, or independently manipulate the media streams to optimize bitrates.

To achieve this, WhatsApp employs a sophisticated, multi-device key distribution mechanism based on advanced cryptographic primitives including the Double Ratchet Algorithm, Triple Diffie-Hellman, Curve25519, AES, and HMAC_SHA256.³⁴ When an initiator starts a voice or video call, their device generates a highly secure, random 32-byte Secure Real-time Transport Protocol (SRTP) master secret specifically for the recipient's device.³⁴ This secret is encapsulated within an incoming call message and transmitted using a client-fanout approach—sending it to every registered device of the recipient via already established pairwise encrypted sessions.³³

In a group call scenario, which handles up to 32 participants for voice and 8 for video, the architecture mathematically prevents the Meta servers from accessing any key material.³⁶ The central server randomly selects one active participant device present in the call. This chosen device generates the SRTP master secret and securely distributes it to all other active devices using individual pairwise encryption channels.³⁵ Every time a user joins or leaves the group call, the cryptographic keys are aggressively reset and redistributed to ensure perfect forward secrecy and to prevent unauthorized interception of ongoing packets by former participants.³⁵

Integrating Artificial Intelligence features into an E2EE environment presents a paradox, as large language models (LLMs) traditionally require server-side access to plaintext data. To resolve this, Meta utilizes Private Processing backed by Trusted Execution Environments (TEEs).³⁷ A TEE is a secure enclave within a server processor that ensures data loaded inside is cryptographically protected from the rest of the system. In WhatsApp, when a user requests an AI summarization, the encrypted message is sent to the TEE, decrypted securely inside the enclave, processed by the AI model, and the result is re-encrypted before being sent back to the user.³⁷ This ensures that neither Meta, WhatsApp, nor the underlying operating system can access the plaintext data, preserving the fundamental security guarantees of the platform.³⁷

Microsoft Teams: Modular Client Re-Architecture

Microsoft Teams represents the evolution of client-side architecture in RTC platforms. Initially built upon an Electron host, the AngularJS framework, and custom HTML/CSS controls, the legacy Teams client suffered from excessive memory consumption and high device resource strain due to the compounding weight of adding extensive collaborative features over the years.³⁸

To resolve this systemic bloat, the architecture was systematically overhauled into a modular framework. The new client entirely abandoned Electron in favor of WebView2, which

significantly reduces the memory footprint and CPU utilization by sharing rendering resources directly with the underlying operating system.³⁸ The user interface transitioned from Angular to React, utilizing the standardized Fluent UI collection for highly optimized, predictable rendering cycles.³⁸

Furthermore, data processing and state management were aggressively extracted from the main UI thread into isolated background worker processes. This decoupling ensures that heavy computational operations—such as computing video layouts, parsing complex calendar payloads, or processing real-time telemetry—do not block the main thread, resulting in a UI that remains highly responsive even during massive group calls.³⁸ Microsoft also invested heavily in network reliability, implementing high-availability clustering configurations for their message delivery services and multi-content delivery networks (CDNs) to isolate failures and ensure 99.99% uptime across geographic regions.³⁶ Advanced features, such as Copilot call delegation and AI-driven Interpreter agents, are deeply integrated into this new event-driven architecture, utilizing targeted messages for private, temporary updates to specific users without polluting the global state.⁴⁰

Architecting Enterprise Systems on Vercel and Supabase

Deploying a system capable of matching the performance, scalability, and feature parity of these enterprise platforms requires a modern, decoupled technology stack. Vercel and Supabase provide a robust foundation for the application layer, user authentication, high-speed signaling, and structured data persistence. However, specialized WebRTC infrastructure must be orchestrated alongside these platforms to handle the complex, stateful media plane.

The Media Layer Limitation and LiveKit Integration

Vercel's serverless architecture, powered by ephemeral, stateless functions, is fundamentally incompatible with the raw networking requirements of a WebRTC Selective Forwarding Unit. Serverless functions are designed for rapid HTTP request-response cycles and automatically terminate after execution to conserve resources.⁴¹ Conversely, WebRTC necessitates long-lived, highly stateful connections operating primarily over UDP ports to handle the continuous, low-latency stream of RTP packets traversing the network.⁴¹

To bridge this architectural gap, developers must integrate a dedicated, WebRTC-compatible media server. When evaluating infrastructure providers like Daily.co, 100ms, and LiveKit, structural differences become apparent.⁴⁴ While Daily.co and 100ms provide excellent managed SDKs, LiveKit has emerged as the premier open-source SFU solution for this paradigm due to its architectural flexibility.⁴⁴ Built natively in Go, LiveKit inherently handles the computational complexities of Simulcast, temporal layer selection, bandwidth estimation, and automated ICE negotiations.¹

In this integrated architecture, Vercel acts as the orchestration and interface layer. When a user requests to join a meeting, the Next.js frontend deployed on Vercel triggers a serverless API route. This route securely communicates with the LiveKit backend using private cryptographic keys to mint a strict JSON Web Token (JWT) access payload.³² The Vercel function returns this token to the client, which then establishes a persistent WebSocket connection to the LiveKit server for internal room signaling, followed immediately by a direct WebRTC connection for continuous media transport.⁴⁸ This separation of concerns allows the web application to scale horizontally and globally on Vercel's edge network while the CPU-intensive media routing scales independently on dedicated compute instances optimized for high network throughput.

Feature Comparison	LiveKit	100ms	Daily.co
Architecture	Open-source, Go-based SFU	Proprietary managed service	Proprietary managed service
Deployment Model	Self-hosted or Cloud	Cloud managed only	Cloud managed only
Use Case Focus	Deep infrastructure control	Rapid deployment, Prebuilt UI	Ease of use, Agentic integrations
Customization	High (Full access to core routing)	Moderate (SDK level)	Moderate (SDK level)

WebRTC Signaling with Supabase Realtime

Before media can flow through LiveKit or any P2P connection, endpoints must exchange critical connection metadata. This process, known as signaling, involves the asynchronous exchange of Session Description Protocol (SDP) objects detailing available media formats and codecs, as well as ICE candidates containing precise network routing pathways.¹

Traditionally, engineers utilized persistent WebSocket frameworks like Socket.io running on dedicated Node.js servers for this task.⁵⁰ However, Socket.io introduces complexity regarding server restarts, race conditions between clients, and subtle state drifts when events arrive out of order.⁵⁰ Supabase Realtime provides a highly scalable, serverless-friendly alternative.⁵⁰ Supabase Realtime operates by deeply integrating with PostgreSQL's Write-Ahead Log (WAL)

and logical replication stream to push database mutations to connected clients with immense speed.⁵²

For WebRTC signaling and meeting coordination, Supabase Realtime offers two distinct, high-performance primitives: Broadcast and Presence.⁵⁴

- **Broadcast:** This feature operates on a classic Publisher/Subscriber pattern. It enables clients to transmit messages (such as ephemeral SDP offers and answers) through a unique channel identifier directly to other subscribed clients.⁵⁵ Critically, Broadcast messages bypass the PostgreSQL database entirely.⁵⁴ This architectural decision is vital for signaling, as writing thousands of transient ICE candidates to a persistent relational database would introduce unacceptable I/O latency and rapidly degrade database performance.
- **Presence:** For maintaining the global, shared state of the meeting—such as tracking who is currently online, who holds the active speaker token, or who has raised their hand—Supabase Realtime utilizes Presence. This engine synchronizes shared state using an eventually-consistent, conflict-free algorithm.⁵⁴ When a user joins or drops offline unexpectedly (due to a severed connection), the Presence engine automatically and reliably updates the state across all connected clients, ensuring the participant roster remains perfectly accurate without requiring constant, inefficient API polling.⁵⁴

Capability	Supabase Realtime	Socket.io
State Synchronization	Native (Presence tracks connected users)	Requires custom implementation
Database Integration	Native (Listens to PostgreSQL WAL)	Disconnected from database
Message Routing	Broadcast bypasses DB for low latency	Handled purely in application memory
Scalability Model	Managed, highly scalable out of the box	Requires manual clustering and Redis adapters

Advanced State Management and Collaborative Features

Enterprise platforms are not merely video conduits; they are defined by their rich collaborative capabilities, such as integrated whiteboards, synchronized document editing, and persistent real-time chat.¹⁹ Implementing these features securely, consistently, and without data corruption across distributed networks requires advanced mathematical data structures.

Conflict-Free Replicated Data Types (CRDTs) form the mathematical foundation of modern collaborative web applications. A prominent library in this ecosystem is Yjs, which resolves concurrent, overlapping edits by multiple users flawlessly without requiring a centralized database locking mechanism.⁵⁸ By pairing Yjs directly with the Supabase Realtime Broadcast primitive, the application can synchronize keystrokes, cursor positions, and whiteboard vector strokes across all participants with millisecond precision.⁵⁴ The raw CRDT binary updates are transmitted rapidly via the ephemeral Broadcast channel, while a periodic, debounced synchronization function writes the finalized document state to the Supabase PostgreSQL database for permanent persistence.⁵⁸

At the user interface level, managing the state of a complex video application built in React (the dominant framework on Vercel) is notoriously difficult. Standard React hooks like `useState` and the Context API are fundamentally insufficient for managing the high-frequency state updates inherent in video calls, such as tracking active speaker volume levels or rapid changes in network quality indicators.⁶¹ Over-reliance on the Context API leads to severe "prop drilling" and performance degradation, as updates to a single deeply nested value force cascading re-renders across the entire component tree.⁶²

To circumvent this architectural bottleneck, large-scale React applications utilize atomic or proxy-based state management libraries like Zustand, Jotai, or Redux Toolkit.⁶² These libraries establish state boundaries and allow discrete UI components to subscribe selectively to specific "slices" of the global state. For example, a single video tile component can subscribe exclusively to the `isMuted` boolean of a specific participant.⁶² This paradigm ensures that the component only re-renders when that specific value changes, rather than re-rendering every time any user in the meeting speaks, drastically improving the client's frame rate, responsiveness, and memory efficiency.⁶²

Database Schema Design and Asynchronous Webhook Pipelines

Persistent data within a video meeting—such as user identities, meeting configurations, deep chat histories, and recording metadata—must be structured highly efficiently within Supabase's PostgreSQL environment.

A normalized relational schema is vital for long-term maintainability. A standard architectural pattern involves separating entities into distinct, interrelated tables: meetings, participants, and messages.⁶⁵ The messages table must link back to the meetings table via strict foreign keys and should optimally utilize JSONB column types to store highly variable metadata, such as file

attachment references, rich text formatting, or specific message types (e.g., system notifications vs. user text).⁶⁵ To transmit raw video frames through a database (a novel approach for specific use cases), tables can employ BYTEA columns for JPEG data alongside sequence integers, leveraging PostgreSQL's logical replication to stream frames to clients via pg-relay mechanisms.⁵³

Security is strictly enforced at the database layer utilizing Supabase's Row-Level Security (RLS). RLS policies ensure zero-trust architecture. Through RLS, the database mathematically guarantees that a user can only query, insert, or modify messages belonging to a specific meeting if their authenticated JWT explicitly proves they are a registered participant of that exact meeting, preventing unauthorized data exfiltration.⁵²

Handling post-meeting workflows, such as processing enormous video recordings, generating AI transcripts, or triggering updates to external CRM systems, requires decoupling heavy, long-running tasks from the user's primary execution path. This is achieved natively through the integration of Supabase Database Webhooks and Edge Functions.⁶⁸

When a meeting concludes and a status flag is updated, or a new message row is inserted, a database trigger automatically fires. Utilizing the PG_NET extension, PostgreSQL executes an asynchronous HTTP POST request directly from the database level to a webhook URL.⁶⁹ This webhook invokes a Supabase Edge Function (running on the highly performant Deno runtime). The Edge Function receives the payload (containing the old and new row data) and executes the required business logic—such as making an authenticated API call to an LLM provider for summarization, or pushing a real-time notification to an iOS device—without blocking the main database thread or causing the frontend client to hang while waiting for a response.⁶⁸ This event-driven architecture directly mirrors the highly resilient, asynchronous processing pipelines utilized by major tech enterprises like Zoom and Microsoft.²⁶

Orchestrating AI Agents in Ephemeral Environments

The integration of artificial intelligence directly into the voice and video pipeline is the current frontier of RTC development. Frameworks such as Pipecat, LiveKit Agents, and the TEN Framework have been developed specifically to orchestrate Large Language Models (LLMs) within real-time WebRTC interfaces.⁷³

These frameworks construct linear pipelines composed of distinct steps: Speech-to-Text (STT) transcription of the incoming audio, processing the text through an LLM, and generating a response via Text-to-Speech (TTS) models before streaming the synthesized audio back into the WebRTC session.⁷³ Architecting this requires a sophisticated "Worker-Job" architecture to maintain resilience.⁷⁴

Deploying these agents securely and efficiently is challenging, as voice AI requires long-running processes to maintain persistent WebSocket connections and process audio streams, which typically violate the strict execution timeouts of standard serverless functions.⁴⁸ Vercel Sandbox solves this architectural challenge by providing on-demand, isolated execution

environments.⁴⁸ When a user initiates a call with an AI agent, the Next.js backend securely provisions a Vercel Sandbox. This sandbox spins up a dedicated instance of the LiveKit agent pipeline, connects to the room, processes the entire conversation dynamically, and is instantly destroyed upon call completion.⁴⁸ This ephemeral design prevents memory leaks, ensures perfect session isolation, and eliminates the need to maintain costly, idle persistent servers.⁴⁸

Network Resilience and Quality of Service Protocols

The internet is fundamentally an unpredictable transmission medium. Network congestion, Wi-Fi signal interference, and hardware bottlenecks lead to inevitable packet loss and severe jitter variations.³ If left unmitigated, dropped UDP packets manifest instantly in the user interface as frozen video frames, heavy artifacting, and robotic, unintelligible audio streams. To combat this physical reality, platforms deploy advanced resilience mechanisms and error correction protocols directly within the WebRTC transmission pipeline.

When a receiving client detects a missing packet via sequence number gaps in the RTP header, it immediately issues a Negative Acknowledgment (NACK) message back to the SFU via the RTCP (RTP Control Protocol) channel. The SFU, which actively maintains a short, rolling buffer of recently transmitted media packets, receives the NACK and swiftly retransmits the specifically missing data.⁷⁶

However, in high-latency or severely congested network environments, the total round-trip time required for a NACK request to be sent, processed, and the packet retransmitted often exceeds the strict real-time playback threshold of the video decoder. In such cases, the recovered packet arrives too late to be rendered and is discarded. To solve this mathematically, WebRTC employs Forward Error Correction (FEC).⁷⁶ FEC algorithms proactively generate redundant parity data alongside the primary media stream. By continuously analyzing the packet loss ratio derived from RTCP receiver reports, the application dynamically modulates the ratio of FEC packets injected into the transport stream.⁷⁷ If a primary packet is lost in transit, the receiving client utilizes the surviving FEC packets to mathematically reconstruct the missing payload instantaneously upon arrival, completely circumventing the need for a time-consuming retransmission request.⁷⁷

For audio streams, where uninterrupted playback is crucial for intelligibility, specific techniques like Redundant Audio Data (RED) are employed. RED encapsulates older, previously transmitted audio frames within the payload of current packets, expanding the packet header.⁷⁹ If an audio packet drops, the subsequent packet arrives containing a lower-quality, redundant copy of the lost audio, allowing continuous playback without requiring algorithmic reconstruction.⁷⁹ In catastrophic scenarios where both primary packets and all layers of redundancy fail to arrive, the client software executes Packet Loss Concealment (PLC). PLC algorithms utilize the surrounding, successfully received audio data to mathematically synthesize a filler signal that mimics the lost audio segment's pitch and frequency, preventing jarring audio dropouts and smoothing the auditory experience for the end user.⁷⁹

Architecting Extensible Plugin Systems

Enterprise platforms generate immense peripheral value and user lock-in through the provision of third-party integrations and developer ecosystems, perfectly exemplified by the Google Workspace Add-ons SDK or the Zoom Video SDK.¹⁹ These expansive ecosystems allow external developers to embed custom applications—ranging from collaborative whiteboards to specialized CRM tools—directly into the meeting interface. Attempting to build a monolithic application containing every possible feature inevitably leads to insurmountable code bloat, tight coupling, and eventual architectural collapse.⁸⁰

Developing a highly sophisticated, enterprise-ready video meeting application on Vercel and Supabase necessitates the implementation of a modular plugin architecture from inception, based heavily on the principles of inversion of control and dependency injection.⁸⁰ Instead of hardcoding third-party features directly into the core application logic, the core platform must be designed as an agnostic execution environment that facilitates plugin life cycles.

Structurally, this is achieved by implementing a robust Event Bus or pipeline architecture within the client application. The core WebRTC state manager is programmed to emit highly standardized lifecycle events, such as `onParticipantJoined`, `onMeetingStarted`, or `onAudioTrackMuted`.⁸² Custom plugins, encapsulated as independent React components or discrete logic modules, can be dynamically loaded and registered to listen to these specific system hooks.⁸² For example, a specialized calendar integration plugin (mirroring Zoom's deep Google Calendar integration) can hook into the `onMeetingScheduled` event to securely push meeting metadata out via OAuth 2.0 authorized APIs without modifying the core scheduling code.³⁰

By adopting the Command Pattern and strict encapsulation, plugins execute their operations through consistent, highly sandboxed interfaces. This defensive programming methodology prevents a faulty or resource-intensive plugin from crashing the underlying WebRTC connections, blocking the main thread, or causing catastrophic memory leaks within the overarching React component tree.⁸⁰

The technological chasm that once separated proprietary, billion-dollar enterprise video conferencing giants from independent developmental frameworks has been fundamentally closed by the advent of edge computing and managed RTC infrastructure. The rigid architectural mandates that define platforms like Google Meet, Zoom, and WhatsApp—specifically the requisite use of Selective Forwarding Units, Simulcast routing, aggressive state isolation, machine learning optimizations, and complex cryptographic fan-out methodologies—are now fully reproducible using modern, decoupled cloud primitives.

By masterfully orchestrating the serverless frontend routing and ephemeral environments of Vercel, the real-time presence engines, persistent database webhooks, and granular security policies of Supabase, and the dedicated, Go-based WebRTC media processing capabilities of SFU infrastructure like LiveKit, developers possess the exact tools required to construct

massively scalable, highly resilient video platforms. Success in this complex domain relies entirely on strict architectural discipline: treating the PostgreSQL database exclusively as the final source of truth for persistent data, utilizing ephemeral broadcast channels for high-frequency signaling and CRDT synchronization, and implementing highly granular, proxy-based state management within the client to preserve processing overhead for the critical, latency-sensitive tasks of media encoding and network error correction.

Works cited

1. How to Integrate WhatsApp Calling API in Your Web App Using WebRTC - Medium, accessed May 9, 2026, <https://medium.com/@arслан.ali1396/how-to-integrate-whatsapp-calling-api-in-your-web-app-using-webrtc-5c073041e819>
2. WebRTC Signaling Server: How it Works?, accessed May 9, 2026, <https://antmedia.io/webrtc-signaling-servers-everything-you-need-to-know/>
3. Building Real-Time P2P Communication: A Deep Dive into WebRTC, ICE, STUN, and TURN, accessed May 9, 2026, <https://akashsahani2001.medium.com/building-real-time-p2p-communication-a-deep-dive-into-webrtc-ice-stun-and-turn-e645492230c5>
4. How to Design a Zoom — Distributed Video Conferencing Architecture, WebRTC, RTP, SFU, accessed May 9, 2026, <https://medium.com/@li.ying.explore/how-to-design-a-zoom-distributed-video-conferencing-architecture-webrtc-rtp-sfu-0a45b3f928d0>
5. System Design Interview: Video Conferencing (Zoom / Google Meet), accessed May 9, 2026, <https://www.techinterview.org/post/3233460400/system-design-video-conferencing/>
6. Inside My Zoom System Design Interview Platform (and What I Learned the Hard Way) | by Kei Zee | Medium, accessed May 9, 2026, <https://medium.com/@kei.zee/inside-my-zoom-system-design-interview-platform-and-what-i-learned-the-hard-way-36b81a8bb54e>
7. WebRTC Architecture Explained: P2P vs SFU vs MCU vs XDN - Red5 Pro, accessed May 9, 2026, <https://www.red5.net/blog/webrtc-architecture-p2p-sfu-mcu-xdn/>
8. How to Design a Video Conferencing System (Zoom Architecture) - Design Gurus, accessed May 9, 2026, <https://www.designgurus.io/blog/design-video-conferencing-system>
9. Manage virtual media streams in Meet Media API | Google Meet ..., accessed May 9, 2026, <https://developers.google.com/workspace/meet/media-api/guides/virtual-streams>
10. WebRTC Simulcast: How Multi-Quality Video Streams Work - BlogGeek.me, accessed May 9, 2026, <https://bloggeek.me/webrtcglossary/simulcast/>
11. How Video Calling Actually Works: WebRTC and the Architecture Behind Real-Time Communication | Ayush Sharma, accessed May 9, 2026, <https://aysh.me/blogs/how-video-calling-works-webrtc>

12. Simulcast Definition: The Tech Behind Efficient Meetings - TrueConf, accessed May 9, 2026, <https://trueconf.com/blog/wiki/simulcast>
13. Scalable Video Conferencing Using SDN Principles - arXiv, accessed May 9, 2026, <https://arxiv.org/html/2503.11649v1>
14. FOSDEM 2022 - Backgrounds are blurry, but the future is clear, accessed May 9, 2026, https://archive.fosdem.org/2022/schedule/event/rtc_webrtc_ai_ml/
15. What algorithm does Zoom use for person segmentation? : r/computervision - Reddit, accessed May 9, 2026, https://www.reddit.com/r/computervision/comments/jnivaw/what_algorithm_does_zoom_use_for_person/
16. Live Background Blur..How Does It Work? | by Anirudh Topiwala | The Startup | Medium, accessed May 9, 2026, <https://medium.com/swlh/live-background-blur-how-does-it-work-4c0491bcf2fa>
17. ML in WebRTC: The noise suppression gold rush - BlogGeek.me, accessed May 9, 2026, <https://bloggeek.me/webrtc-noise-suppression/>
18. How we built a real-time, client-side noise suppression library without server dependencies, accessed May 9, 2026, <https://www.datadoghq.com/blog/engineering/noise-suppression-library/>
19. Google Meet SDK and API overview, accessed May 9, 2026, <https://developers.google.com/workspace/meet/overview>
20. Meet add-on concepts - Google for Developers, accessed May 9, 2026, <https://developers.google.com/workspace/meet/add-ons/guides/concepts>
21. Meet add-ons SDK for Web overview - Google for Developers, accessed May 9, 2026, <https://developers.google.com/workspace/meet/add-ons/guides/overview>
22. Google Meet Add-ons SDK is now generally available, accessed May 9, 2026, <https://workspaceupdates.googleblog.com/2024/09/google-meet-add-ons-sdk-is-now-available.html>
23. Collaborate using a Meet add-on - Google for Developers, accessed May 9, 2026, <https://developers.google.com/workspace/meet/add-ons/guides/collaborate-in-the-add-on>
24. How to enable cross-platform experiences with Google Meet - YouTube, accessed May 9, 2026, <https://www.youtube.com/watch?v=3mTbD4jBR8U>
25. Learn how to use Chat with Google Meet, accessed May 9, 2026, <https://support.google.com/meet/answer/9308979?hl=en>
26. System Design of Zoom. Zoom is a cloud-based video... | by Santosh P. | Medium, accessed May 9, 2026, <https://medium.com/@lazygeek78/system-design-of-zoom-9f0b945a5636>
27. GUIDETOZOOMBREAKOUTRO, accessed May 9, 2026, <https://www.esquiresolutions.com/wp-content/uploads/2025/09/2025-EDS-Guide-to-ZOOM-Breakout-Rooms.pdf>
28. Zoom breakout rooms: Create focused discussions, accessed May 9, 2026, <https://www.zoom.com/en/products/virtual-meetings/features/breakout-rooms/>
29. Using the Google Connector - Zoom Support, accessed May 9, 2026, https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0078373
30. Managing the calendar and contacts integration - Zoom Support, accessed May

9, 2026,

https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0069503

31. Using calendar and contacts integration - Zoom Support, accessed May 9, 2026, https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0068615
32. Build a video conferencing app with the Zoom Video SDK - Zoom Developer Blog, accessed May 9, 2026, <https://developers.zoom.us/blog/build-a-video-conferencing-app-with-the-zoom-video-sdk/>
33. WhatsApp Encryption Technical Overview | PDF - Scribd, accessed May 9, 2026, <https://www.scribd.com/document/628464023/328619852-1611854775910507-3217786758896242477-n-1>
34. WhatsApp's End to End Encryption, How does it work? | by Amit Panghal - Medium, accessed May 9, 2026, <https://medium.com/@panghalamit/whatsapp-s-end-to-end-encryption-how-does-it-work-80020977caa0>
35. How WhatsApp enables multi-device capability - Engineering at Meta - Facebook, accessed May 9, 2026, <https://engineering.fb.com/2021/07/14/security/whatsapp-multi-device/>
36. How WhatsApp Works - Architecture Deep Dive on 100 Billion Messages, accessed May 9, 2026, <https://getstream.io/blog/whatsapp-works/>
37. Private Processing for WhatsApp Overview - Meta AI, accessed May 9, 2026, <https://ai.meta.com/static-resource/private-processing-technical-whitepaper>
38. Microsoft Teams: Advantages of the new architecture | Microsoft ..., accessed May 9, 2026, <https://techcommunity.microsoft.com/blog/microsoftteamsblog/microsoft-teams-advantages-of-the-new-architecture/3775704>
39. Microsoft Teams: Building a Foundation for the Future, accessed May 9, 2026, <https://techcommunity.microsoft.com/blog/microsoftteamsblog/microsoft-teams-building-a-foundation-for-the-future/4090393>
40. What's New in Microsoft Teams | April 2026, accessed May 9, 2026, <https://techcommunity.microsoft.com/blog/microsoftteamsblog/whats-new-in-microsoft-teams--april-2026/4515907>
41. Recommended way to implement a SFU with web rtc - Google Groups, accessed May 9, 2026, <https://groups.google.com/g/discuss-webrtc/c/CzWufCOdjaq>
42. Vercel Functions, accessed May 9, 2026, <https://vercel.com/docs/functions>
43. You don't need a WebRTC server for your voice agents - Daily.co, accessed May 9, 2026, <https://www.daily.co/blog/you-dont-need-a-webrtc-server-for-your-voice-agents/>
44. Livekit vs Daily.co for Voice AI — Which Is Better? - AnveVoice, accessed May 9, 2026, <https://anvevoice.app/faq/livekit-vs-daily-co-for-voice-ai>
45. 8 Best LiveKit Alternatives - GetStream.io, accessed May 9, 2026, <https://getstream.io/blog/livekit-alternatives/>
46. 100ms vs LiveKit: Which Platform Suits You Best? - Video SDK - VideoSDK, accessed May 9, 2026, <https://www.videosdk.live/100ms-vs-livekit>

47. Use Case: Video Conferencing - LiveKit, accessed May 9, 2026,
<https://livekit.com/use-cases/video-conferencing>
48. How to build an on-demand voice agent with Vercel Sandbox, accessed May 9, 2026,
<https://vercel.com/kb/guide/how-to-build-an-on-demand-voice-agent-with-vercel-sandbox>
49. Building a Real-Time Video Consultation Platform with LiveKit, Supabase, React & Flutter, accessed May 9, 2026,
<https://medium.com/@zendeaniket524/building-a-real-time-video-consultation-platform-with-livekit-supabase-react-flutter-1a12dc4498e3>
50. Solo dev: Switched from Socket.io to Supabase Realtime for my turn-based game. Way simpler. - Reddit, accessed May 9, 2026,
https://www.reddit.com/r/Supabase/comments/1rgj3po/solo_dev_switched_from_socketio_to_supabase/
51. Supabase Realtime vs Socket.io for React Native (2026) | AI App Factory | PEC Camp, accessed May 9, 2026,
<https://productengineer.info/camp/en/ai-app-factory/compare/supabase-realtime-vs-socket-io>
52. Realtime: Broadcast from Database - Supabase, accessed May 9, 2026,
<https://supabase.com/blog/realtime-broadcast-from-database>
53. Video Conferencing with Postgres - PlanetScale, accessed May 9, 2026,
<https://planetscale.com/blog/video-conferencing-with-postgres>
54. Realtime: Multiplayer Edition - Supabase, accessed May 9, 2026,
<https://supabase.com/blog/supabase-realtime-multiplayer-general-availability>
55. Realtime | Supabase Docs, accessed May 9, 2026,
<https://supabase.com/docs/guides/realtime>
56. Realtime - Broadcast | Supabase Features, accessed May 9, 2026,
<https://supabase.com/features/realtime-broadcast>
57. How Video Conferencing Works: Architecture, Workflow & Meet Hour Insights, accessed May 9, 2026,
<https://meethour.io/blog/tech-talks-%26-guides/how-video-conferencing-works-everything-you-need-to-know-about-video-call-technology>
58. A Yjs provider that uses Supabase Realtime for synchronization. Feedback / suggestions welcome. - Reddit, accessed May 9, 2026,
https://www.reddit.com/r/Supabase/comments/10u96rr/a_yjs_provider_that_uses_supabase_realtime_for/
59. How to synchronize your Liveblocks Yjs document data to a Supabase Postgres database, accessed May 9, 2026,
<https://liveblocks.io/docs/guides/how-to-synchronize-your-liveblocks-yjs-document-data-to-a-supabase-postgres-database>
60. TipTap / YJS Collaborative Editing with Supabase Realtime #27105 - GitHub, accessed May 9, 2026, <https://github.com/orgs/supabase/discussions/27105>
61. React State Management Explained: What Works at Scale - YouTube, accessed May 9, 2026, <https://www.youtube.com/watch?v=Pfm895NF1rE>
62. Mastering React State Management at Scale in 2025 - DEV Community, accessed

May 9, 2026,

https://dev.to/ash_dubai/mastering-react-state-management-at-scale-in-2025-52e8

63. [AskJS] How do you efficiently manage state in large-scale React applications without Redux? : r/javascript - Reddit, accessed May 9, 2026, https://www.reddit.com/r/javascript/comments/1dk4usz/askjs_how_do_you_efficiently_manage_state_in/
64. State Management in React: What You Should Know - Mindpath Tech, accessed May 9, 2026, <https://www.mindpathtech.com/blog/state-management-in-react/>
65. Chat schema for PostgreSQL and MongoDB - Database Administrators Stack Exchange, accessed May 9, 2026, <https://dba.stackexchange.com/questions/306757/chat-schema-for-postgresql-and-mongodb>
66. PostgreSQL: Database structure for Chat Conversation - Stack Overflow, accessed May 9, 2026, <https://stackoverflow.com/questions/52662334/postgresql-database-structure-for-chat-conversation>
67. Subscribing to Database Changes | Supabase Docs, accessed May 9, 2026, <https://supabase.com/docs/guides/realtime/subscribing-to-database-changes>
68. Edge Functions | Supabase Docs, accessed May 9, 2026, <https://supabase.com/docs/guides/functions>
69. Database Webhooks | Supabase Docs, accessed May 9, 2026, <https://supabase.com/docs/guides/database/webhooks>
70. Supabase Database Webhooks for Local Development | by Owen Thomson - Medium, accessed May 9, 2026, <https://medium.com/@owenrthomson/supabase-database-webhooks-for-local-development-38d6eba435c5>
71. Using Supabase Edge Functions + Database Webhooks to push notifications straight to your iPhone - Reddit, accessed May 9, 2026, https://www.reddit.com/r/Supabase/comments/1s1ly8r/using_supabase_edge_functions_database_webhooks/
72. Automating Client Onboarding with Supabase Edge Functions and Webhooks, accessed May 9, 2026, <https://dev.to/syntora/automating-client-onboarding-with-supabase-edge-functions-and-webhooks-1g3j>
73. RealTime AI Agents frameworks comparison: LiveKit, Pipecat and TEN | by Gustavo Garcia, accessed May 9, 2026, <https://medium.com/@ggarciabernardo/realtime-ai-agents-frameworks-bb466cb2a09>
74. LiveKit Agents Architecture: Building Real-Time AI Agents for Production - Moravio, accessed May 9, 2026, <https://www.moravio.com/blog/livekit-agents-for-building-real-time-ai-agents>
75. 8 Common Zoom Network Issues & How to Fix Them - Obkio, accessed May 9, 2026, <https://obkio.com/blog/zoom-network-issues/>
76. WebRTC: Forward Error Correction | Talk at VideoTech 2022, accessed May 9,

- 2026,
<https://vtconf.com/en/archive/2022/talks/c96e2252b93448e0a91301b863b0b193/>
77. HANDLING PACKET LOSS IN WEBRTC - Google Research, accessed May 9, 2026,
<https://research.google.com/pubs/archive/41611.pdf>
 78. RFC 8854: WebRTC Forward Error Correction Requirements, accessed May 9, 2026, <https://www.rfc-editor.org/rfc/rfc8854.pdf>
 79. Media Resilience in WebRTC - GetStream.io, accessed May 9, 2026,
<https://getstream.io/resources/projects/webrtc/advanced/media-resilience/>
 80. Plugin Architecture in Web Apps (Examples or Code Snippets?) - Stack Overflow, accessed May 9, 2026,
<https://stackoverflow.com/questions/10763006/plugin-architecture-in-web-apps-examples-or-code-snippets>
 81. How I Created my First Plugin-System - DEV Community, accessed May 9, 2026,
<https://dev.to/trendschau/how-i-created-my-first-plugin-system--400>
 82. How to Design a Plugin-Based Architecture in React for Modular Systems? - Reddit, accessed May 9, 2026,
https://www.reddit.com/r/reactjs/comments/1hmk30p/how_to_design_a_pluginbased_architecture_in_react/